LangChain — Quick Learning Notes

1. What is LangChain?

LangChain is a framework for building applications around LLMs. Instead of calling a model alone, you can connect prompts, models, parsers, Python functions, retrievers, tools and other components into workflows.

Mental model: LangChain is mainly about **composing components into pipelines**.

2. Basic LLM workflow

```
Input → Prompt → Model → Output Parser → Final result
```

Prompt prepares the instruction. **Model** generates the response. **Parser** processes the response into the form you need.

3. Runnables

Runnables are composable building blocks that can be connected into workflows.

Sequence:

```
prompt | model | parser
```

The same flow can be written explicitly as RunnableSequence(prompt, model, parser).

Parallel: RunnableParallel sends the same input through multiple Runnables and combines their results.

Passthrough: returns its input unchanged.

Lambda: RunnableLambda turns a normal Python function into a Runnable.

```
Example: RunnableLambda(lambda x: len(x.split()))
```

Branch: RunnableBranch chooses a path according to a condition.

```
Example: if len(report.split()) > 200 → summarize; otherwise → return the report.
```

4. Output handling

StrOutputParser → extracts text from the model response.

PydanticOutputParser → parses/validates the response against a Pydantic schema.

Structured output → when the model/provider supports it, a schema can be supplied directly with with_structured_output().

Prompt instructions tell the model what/how to produce. A parser actually processes the returned response.

5. What you have practiced

PromptTemplate → ChatGroq → StrOutputParser → RunnableSequence → RunnableParallel → RunnableLambda → RunnablePassthrough → RunnableBranch → PydanticOutputParser.

LangChain Components — RAG & Data

1. Prompt Templates

PromptTemplate creates reusable prompts with variables that are filled when the chain is invoked.

```
PromptTemplate(template='Write about {topic}', input_variables=['topic'])
```

2. Chat Models / LLMs

The model generates the response. LangChain provides integrations for providers such as Groq, Hugging Face, Google and others.

```
model = ChatGroq(model=os.getenv('groq_model'))
```

3. Document Loaders

Document loaders bring external data into LangChain's Document representation. TextLoader can load a .txt file.

```
loader = TextLoader('rag.txt')
docs = loader.load()
```

Document: page_content contains the loaded text; metadata contains information such as the source path.

4. RAG Components

Component	Purpose
Document Loader	Loads documents/data.
Text Splitter	Breaks large documents into smaller chunks.
Vector Database	Stores embeddings for similarity search.
Retriever	Finds relevant chunks for a query.

5. Typical RAG flow

```
Documents → Load → Split → Embed → Store vectors → Retrieve relevant chunks → Give context to LLM → Answer
```

The retriever supplies relevant external context to the LLM, helping the response stay grounded in the provided knowledge.

6. Pydantic + Output Parsing

A Pydantic model defines the desired structure and data types. For example:

```
class Summary(BaseModel):
summary: str
key_points: list[str]
word_count: int
```

The field types define structure; they do not automatically enforce requirements such as 'exactly 200 words'. That requirement belongs in the prompt or needs separate validation.

7. Final mental model

LangChain: compose LLM application components.
Runnables: sequence, parallelize, transform, pass through or branch.
Parsers: handle model outputs.
RAG: load → split → retrieve → provide context → generate.